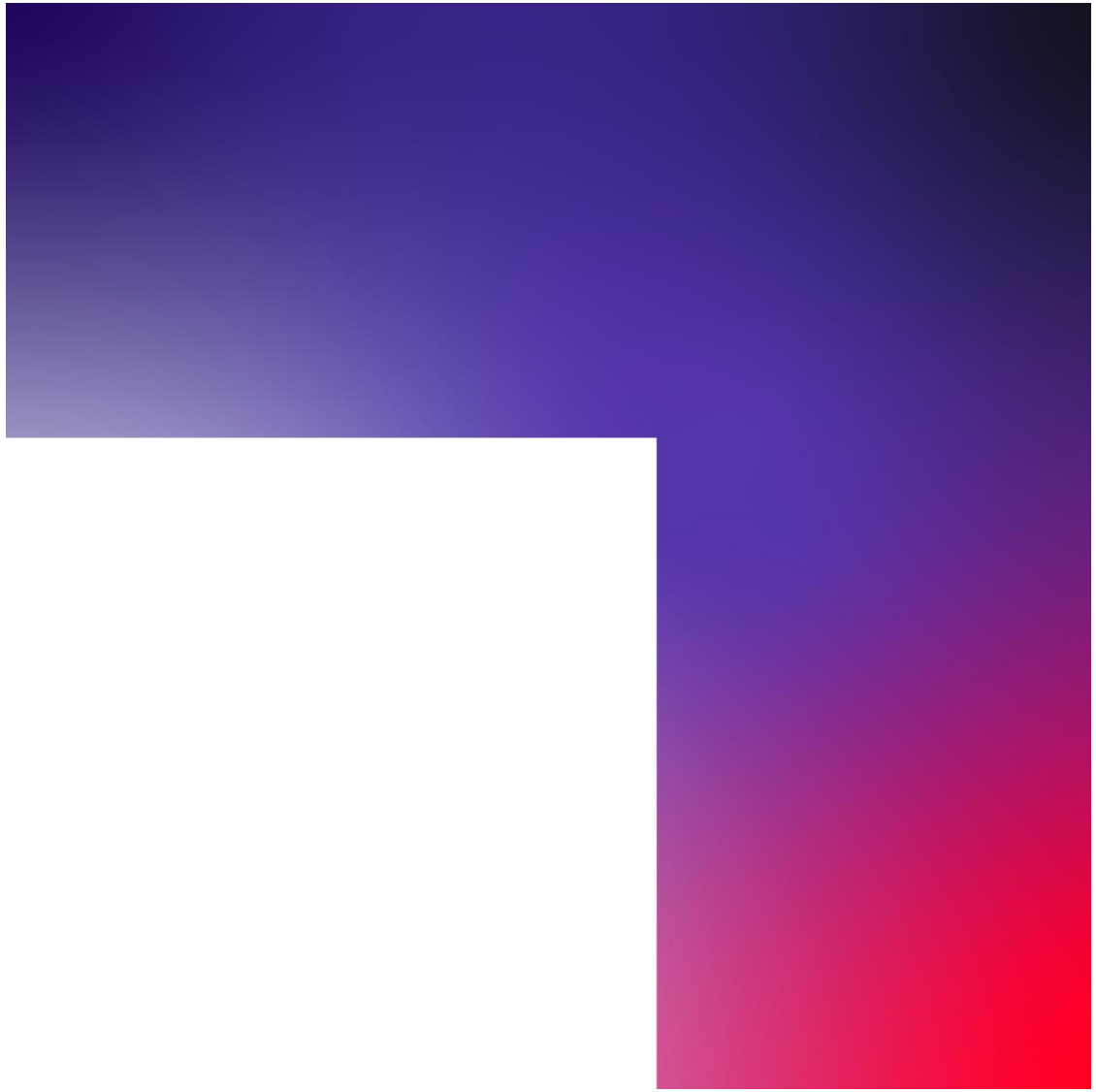
Architecture-aware API testing

exploring of programmatic interfaces

Maaret Pyhäjärvi

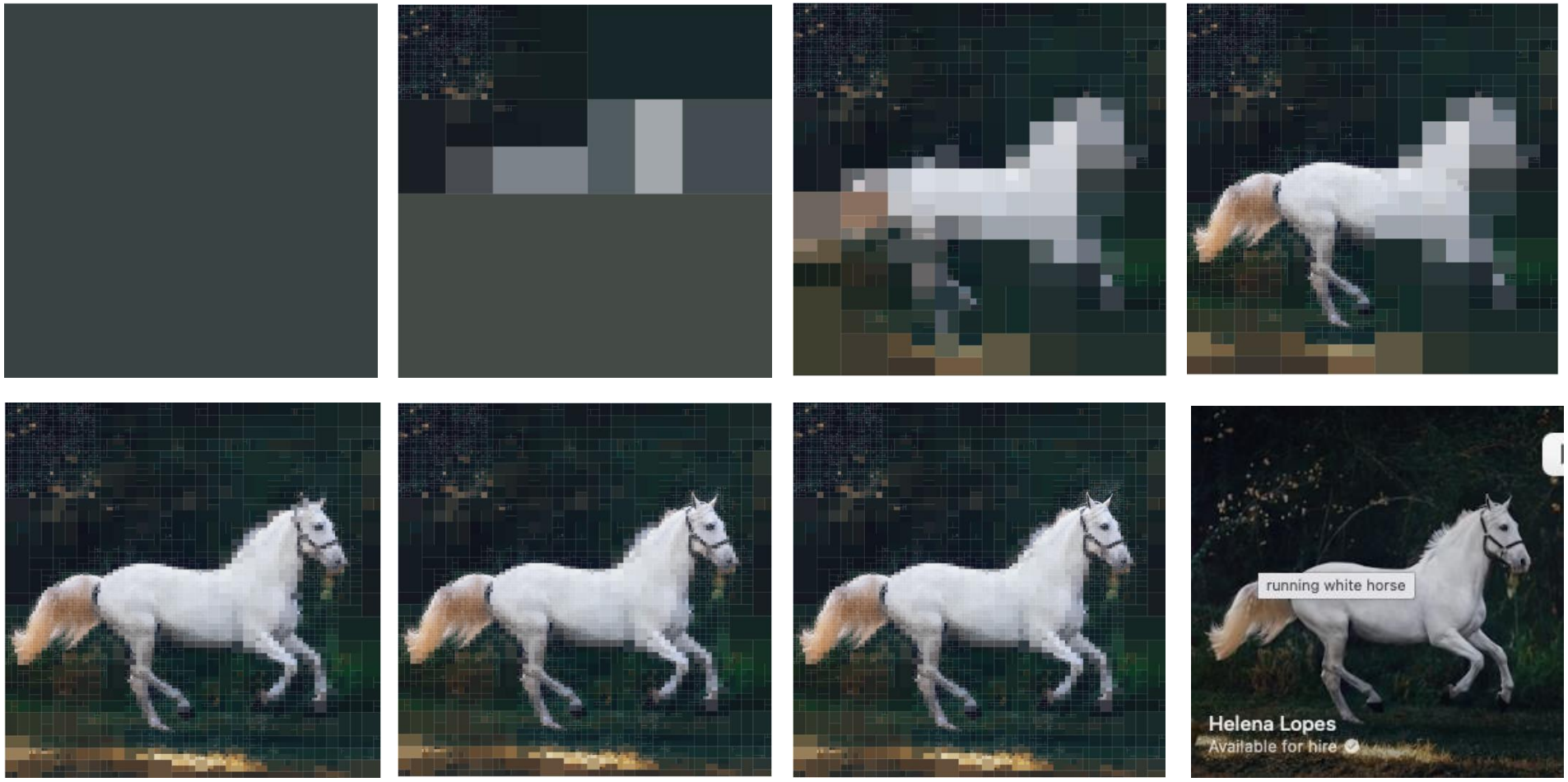
May 2026



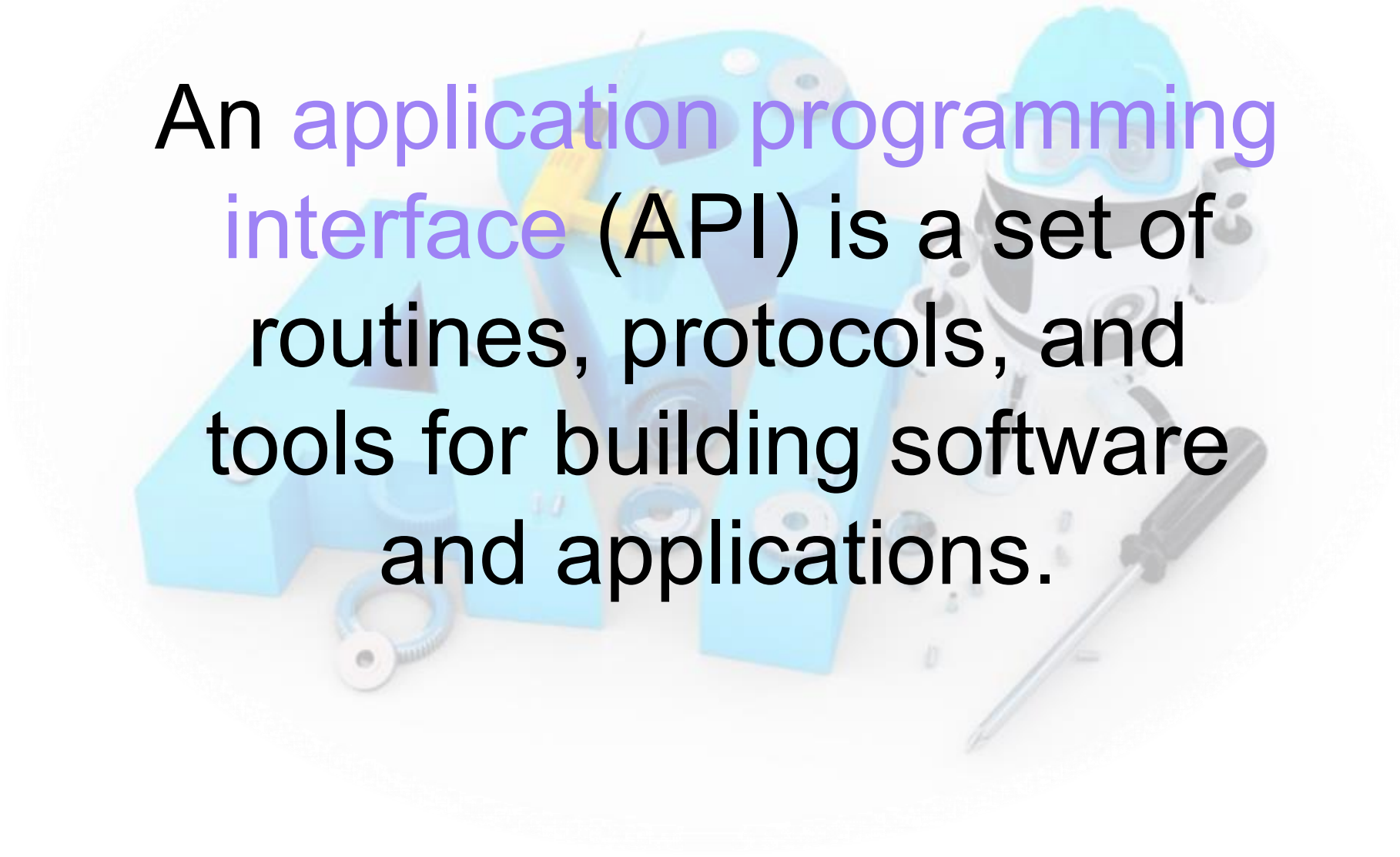
https://github.com/QE-at-CGI-FI/architecture-aware-api-testing

ABOUT TESTING

Search for information that matters



Avoid confidently incorrect judgement



An application programming interface (API) is a set of routines, protocols, and tools for building software and applications.

LEARNING TARGET

Software error taxonomy and other test target APIs

Docs

https://qe-at-cgi-fi.github.io/architecture-aware-api-testing/

Run locally by downloading

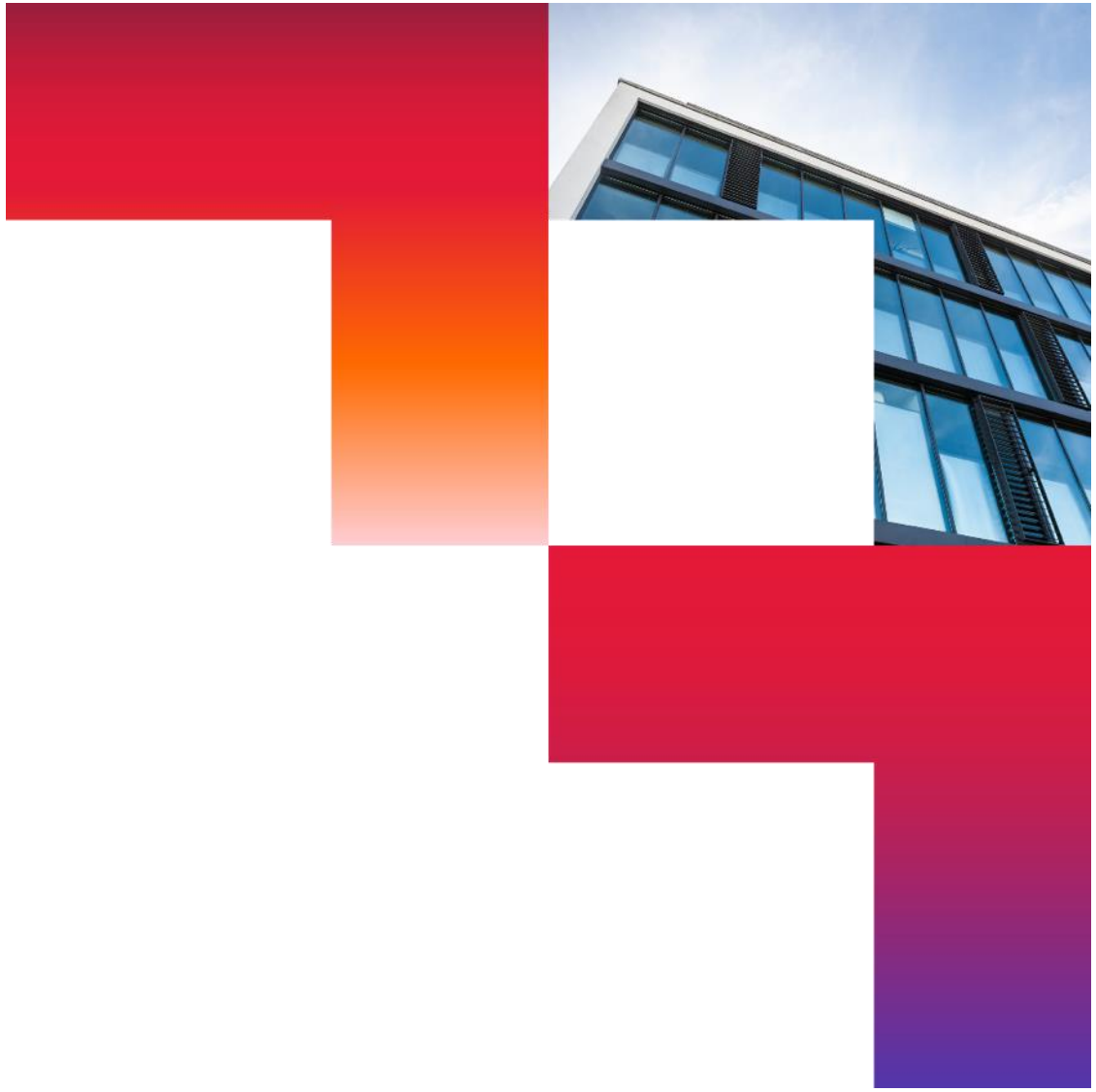
https://github.com/QE-at-CGI-FI/architecture-aware-api-testing

Takeaways to work towards

01 There is no neutral tool. Each approach has an idea about what matters, and the aware tester picks deliberately.

02 You cannot test an API effectively if you treat it as a black box by default. Informed exploration beats uninformed coverage

03 The checklist as a printed/digital reference, plus the experience of having used it under real conditions.



Session 1

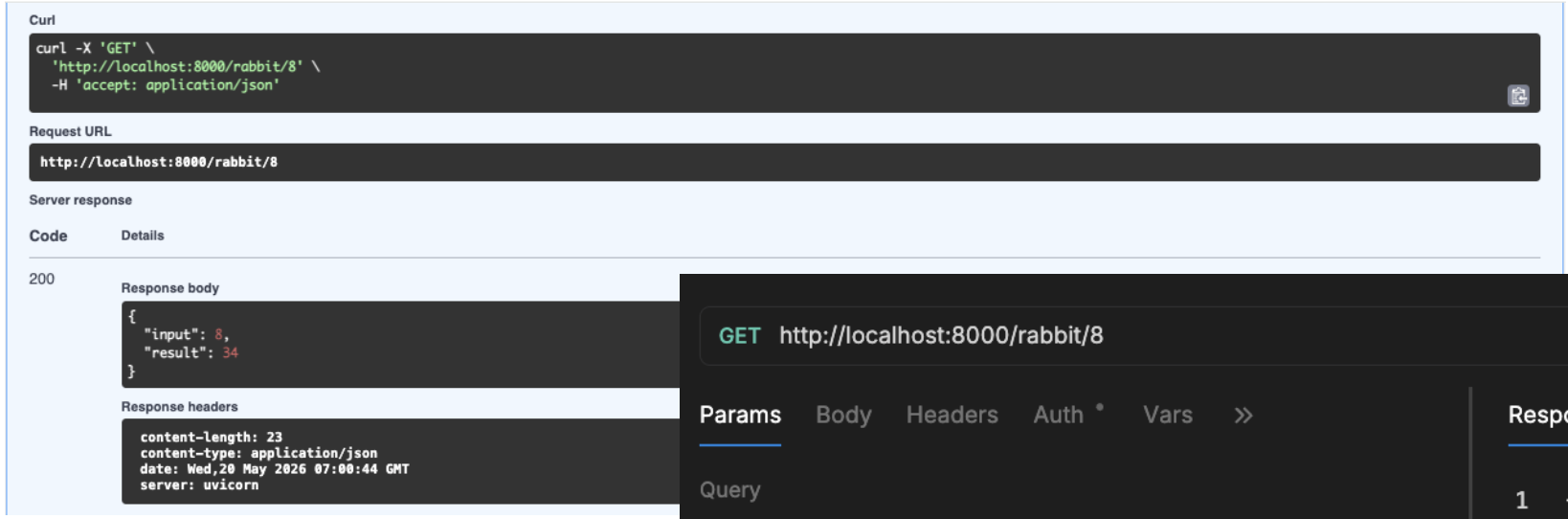
Code vs. Bruno vs. Schemathesis

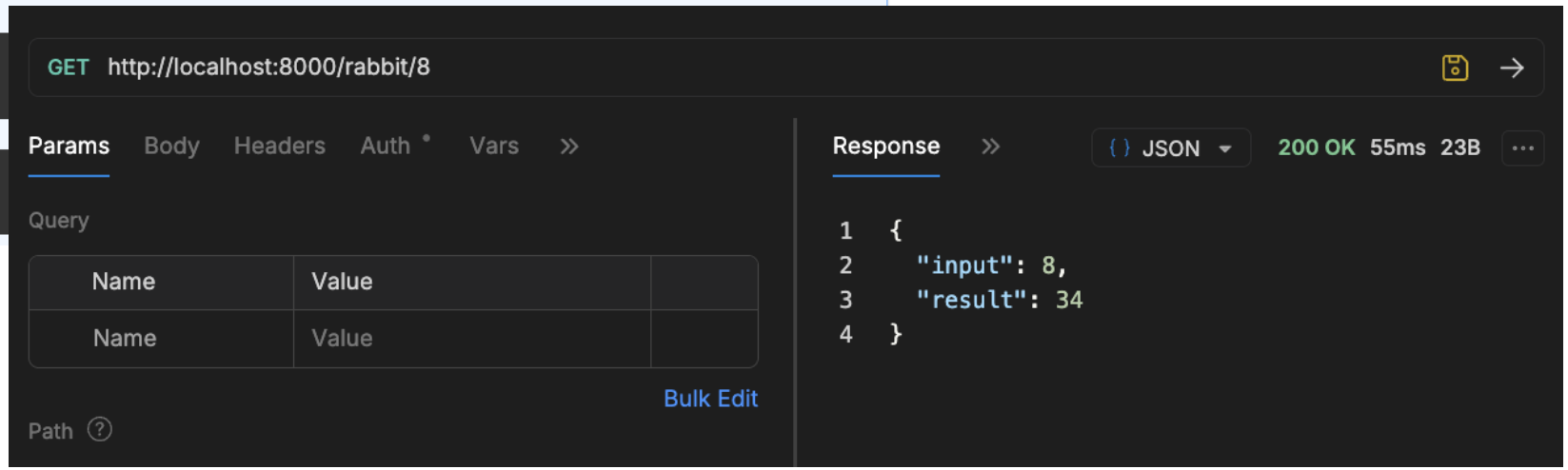
Does your API exploration tool constrain or expand your testing?

HANDS-ON 1

Exercise: Explore one API - rabbit

```python
def test_rabbit_one():
    data = requests.get(f"{BASE_URL}/rabbit/{8}").json()
    assert data == {'input': 8, "result": 34}
```





HANDS-ON 2

Exercise: Explore with three tools

Participants explore the same API using all three approaches in short rotations, then compare notes. What did each approach make easy to find? What did each approach make easy to miss?

Bruno (collection-based, git-friendly exploration) — What does it mean to treat API exploration as a first-class, version-controlled artifact? How does working in a dedicated API client shape what you notice and what you overlook?

Code (programmatic requests via scripts, libraries, or test frameworks) — When you write code to call an API, you gain precision and composability. But do you lose the serendipity of manual exploration?

Schemathesis (properties of APIs to contract-first thinking) — Starting from the schema means starting from the promise. What does the API say it does, and how does that frame what you choose to test? What are properties common to all APIs?

There is no neutral tool. Each approach has an idea about what matters, and the aware tester picks deliberately.

Session 2:

Awareness of What's Behind the API

If you don't know the architecture, how do you know what to test?

HANDS-ON 3

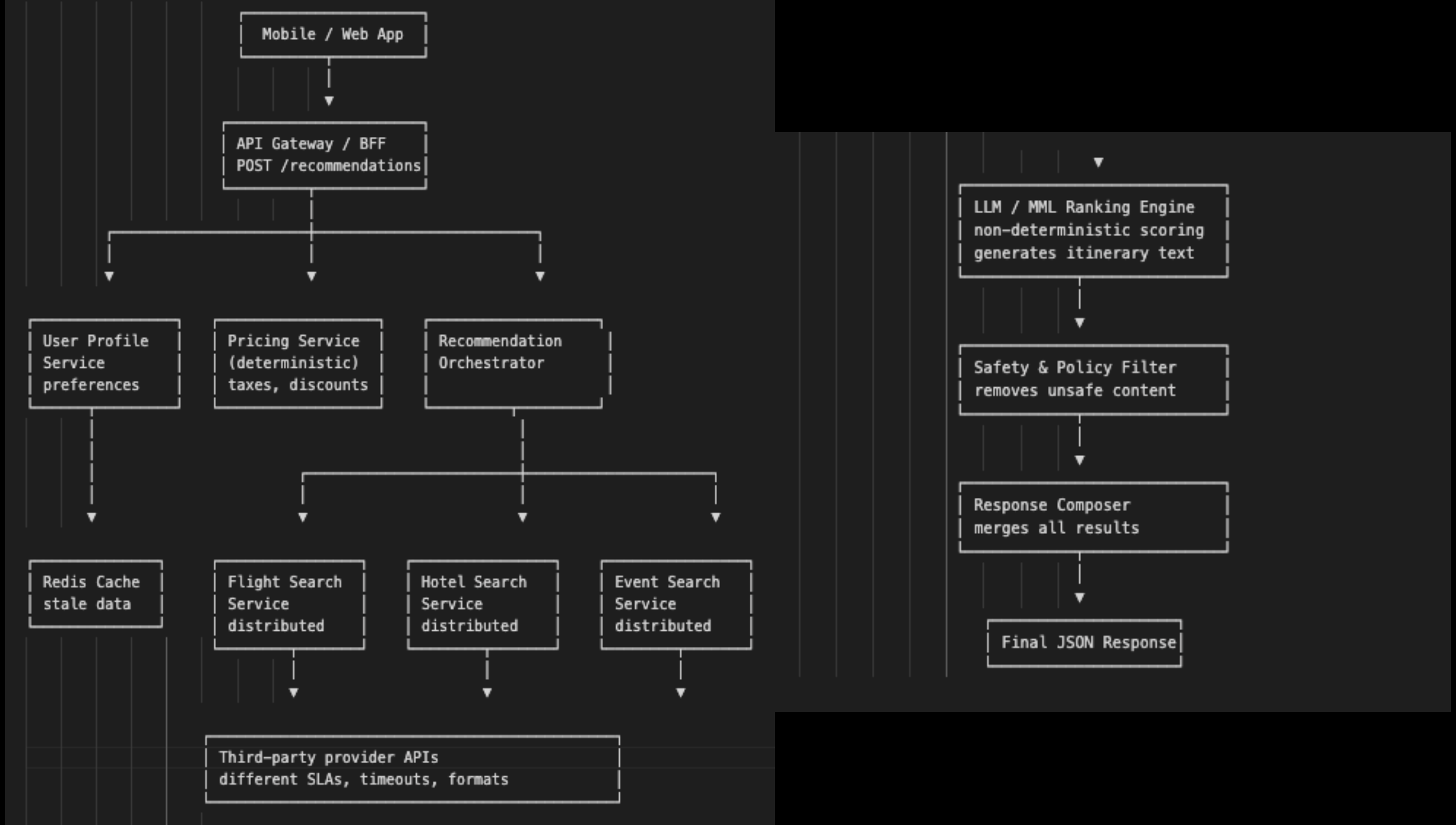
Architectures

Given an architecture diagram of a realistic system, participants map where they would focus exploratory testing and why. Groups compare strategies and discover the perspectives they missed.

The endpoint is a façade. What sits behind it? A direct database call? An orchestration layer coordinating five services? A queue that processes asynchronously? Each of these architectures produces a different failure profile.

Methods are APIs too. The boundary between a caller and a callee is an interface with assumptions, contracts, and failure modes — whether it's an HTTP endpoint, a function signature, a message on a bus, or an event trigger. Testers who see this think in terms of boundaries, not just endpoints.

Architecture patterns change risk. A synchronous call to a monolith fails differently than an async message through a broker. Caching layers hide staleness. API gateways apply policies you didn't write. Rate limiters, circuit breakers, and retry logic all live between your request and the thing that does the work.


Mobile / Web App
API Gateway / BFF
POST /recommendations
User Profile Service
preferences
Pricing Service
(deterministic)
taxes, discounts
Recommendation Orchestrator
Redis Cache
stale data
Flight Search Service
distributed
Hotel Search Service
distributed
Event Search Service
distributed
Third-party provider APIs
different SLAs, timeouts, formats
LLM / MML Ranking Engine
non-deterministic scoring
generates itinerary text
Safety & Policy Filter
removes unsafe content
Response Composer
merges all results
Final JSON Response

You cannot test an API effectively if you treat it as a black box by default. Informed exploration beats uninformed coverage.

Session 3:

The Perspectives Checklist

What should you always consider — and what do you keep forgetting?

HANDS-ON 4

Checklists of perspectives

Participants apply the checklist to a live API, working in pairs. Each pair picks two perspectives to explore deeply, then presents their findings to the group. The goal is not coverage — it's demonstrating how a perspective shift changes what you find.

A thing that can be called can be called with just about anything. If an endpoint accepts input, someone will send it something you didn't expect. Malformed payloads, wrong types, empty bodies, enormous bodies, unexpected encodings, duplicate parameters — the call surface is the attack surface.

Security controls have API impact. Authentication, authorization, rate limiting, input validation, CORS policies, token expiry — these aren't just security concerns. They shape what the API does and doesn't allow, and they interact with functional behavior in ways that surprise people. A tester who ignores security controls is testing a version of the API that doesn't exist in production.

State and sequence matter. APIs aren't stateless just because HTTP is. What happens when you call things out of order? What happens when you call the same thing twice? What about concurrent calls that compete for the same resource?

Error responses are a feature. What does the API tell you when things go wrong? Too much information leaks implementation details. Too little leaves callers guessing. Inconsistent error shapes break clients.

Contracts drift. The documentation says one thing, the schema says another, and the implementation does a third. Testing the gaps between these is some of the highest-value work a tester can do.

Environment and configuration shift behavior. The same API in dev, staging, and production may behave differently because of feature flags, data volumes, downstream dependencies, or infrastructure configuration.

Observable is testable. Does the API produce the logs, metrics, and traces that someone will need when debugging at 2 AM? If you can't observe it, you can't operate it.

The checklist as a printed/digital reference, plus the experience of having used it under real conditions.

DX

Developer Experience

Building it the First Time

Debugging and Maintainin g

REST APIs

Foundational information

Nouns

Verbs

“CRUD”

Methods

HTTP GET
HTTP POST
HTTP PUT
HTTP DELETE
HTTP PATCH

Status Codes

1xx: Informational
2xx: Success
3xx: Redirection

4xx: Client Error
5xx: Server Error

Verbs
Authorization/authentication
Data
Errors
Responsiveness

http://qa-matters.com/2016/07/30/vader-a-rest-api-test-heuristic/

REST APIs

Exploration checklist

Working with limited understanding: Focus!

Calls and Operations. Inputs and Outputs. Exceptions.

Target of your testing vs. the environment it depends on

Specific user with a specific purpose

Building it the First Time

Debugging and Maintaining

Why would anyone use this?

Think lifecycle

Versioning and Deprecation

Google for Concepts

like “Conservative Overloading Strategy”

Fast-track your understanding: collaborate

Documentation Matters a Lot for APIs

Explore to create documentation

Some patterns become visible with repetition

Existing automation allows for exploration reach

Creating automation forces exploration of details

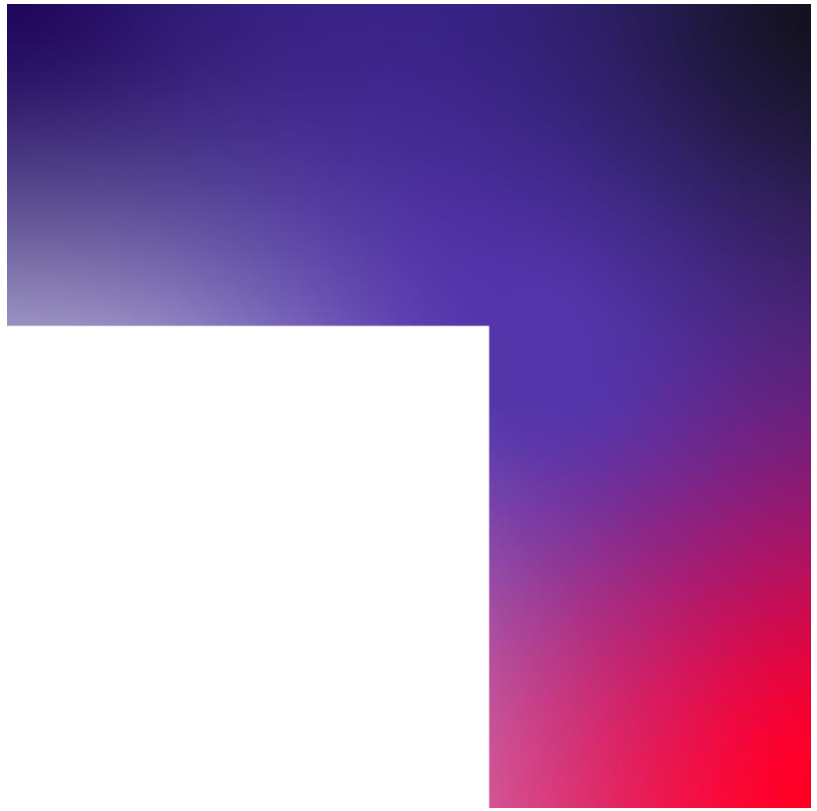
Failing automation is invitation to explore

Insights you can act on

Founded in 1976, CGI is among the largest IT and business consulting services firms in the world.

We are insights-driven and outcomes-focused to help accelerate returns on your investments. Across hundreds of locations worldwide, we provide comprehensive, scalable and sustainable IT and business consulting services that are informed globally and delivered locally.

cgi.com



CGI